

프로젝트명	사내 모바일 앱스토어 시스템 Spring Boot 3.x 전환 및 현대화 프로젝트		
연계/소속	그룹사 운영팀 / 포커스원	개발 인원	1명 (개발·운영 전 과정 수행)
주요 업무	백엔드 개발 및 시스템 마이그레이션 담당		
담당 역할	프레임워크 마이그레이션 총괄, 멀티모듈 아키텍처 재설계, 인증·보안 체계 전환(Spring Security 6.x · 사내 SSO 연동), 운영 배포 및 안정화		
기술 스택	Java 17, Spring Boot 3.3.2, Spring Security 6.x, MyBatis, JPA/Hibernate, MariaDB, Gradle 8.8, Git		
업무 기간	2025.11 ~ 2026.06 (약 7개월) · 운영 배포 완료, 이후 운영·고도화 지속 중		
개발 환경	IntelliJ IDEA, Git / 운영: Linux(페쇄망), 개발: 페쇄망 VDI(Windows)		

S그룹 임직원 약 11만 명이 사용하는 사내 모바일 앱스토어의 레거시 백엔드 (**Java 1.7 / 구형 Spring Framework**)를 **Java 17 / Spring Boot 3.3.2** 기반으로 전환하고, **2026년 6월 운영 배포까지 완료**했습니다. 단순 버전 이식이 아닌 **아키텍처 재설계 · 보안 강화 · 코드 품질 개선**을 병행했습니다.

전환 과정에서 발생한 호환성 이슈를 유형별로 분류·기록하며 전량 해결했고, 페쇄망이라는 제약 속에서 **체계적인 로깅 전략과 계층별 검증 절차**를 직접 설계해 서버 접속 없이 **로그만으로 원인을 특정할 수 있는 환경**을 만들었습니다. 이후 신규 로그인 보안 시스템 설계, **MyBatis + JPA 하이브리드 아키텍처** 구성, **Spring Security 6.x 기반 인증 구조 전환**과 사내 SSO(SAML) 인증 개편까지 수행했습니다. 개발 전 10년의 고객 응대 경험은 장애 상황에서 사용자 영향을 먼저 살피는 습관으로 이어지고 있습니다.

53건

호환성 이슈 전량 해결
유형별 이슈 분류 기록 기준

3일→1일

버그 수정 주기 단축
이슈 발생~해결 작업 기록 기준

28%

빌드 시간 단축
전환 전후 빌드 로그 비교 기준

상세 내용

1. 계층별 단계 전환 전략 — 여러 계층이 한꺼번에 깨지면 원인을 찾을 수 없다

PROBLEM

Java 클래스 151개 · Mapper XML 19개 · JSP 66개 규모의 시스템에서 Jakarta EE 전환, Spring Security 재작성, MyBatis 설정 교체, JSP 뷰 연동까지 동시에 손을 대면 어디서 오류가 났는지 추적이 불가능해집니다. 특히 코드 수정 후 페쇄망 서버에 배포해야만 확인이 가능한 환경이었기 때문에, 원인 불명의 오류 하나가 수 시간의 낭비로 이어지는 구조였습니다.

ACTION

처음에는 문서에 나온 마이그레이션 가이드 순서대로 따라가려 했습니다. 그런데 실제로 시작해보니 여러 계층이 한꺼번에 깨지면 어디가 원인인지 알 수 없다는 걸 초반에 바로 체감했습니다.

"빌드와 기동이 되는 상태를 절대 깨지 않는다"는 원칙을 세우고, **인프라(빌드·DB 연결) → 보안·인증 → 비즈니스 로직 → 뷰** 순으로 한 번에 한 계층씩 진행했습니다. 한 번에 바꾸는 범위가 작을수록 전체 속도가 오히려 빨라진다는 것을 초반 시행착오로 확인했기 때문입니다.

페쇄망 제약을 고려해 로깅 인프라를 가장 먼저 구축했습니다. ViewResolver TRACE · Spring Security DEBUG 수준의 로그 레벨을 설정하고, 클라이언트 IP와 세션ID가 모든 로그에 자동 포함되도록 **커스텀 Logback Converter 기반 로깅 구조**를 직접 구현했습니다. 테스트 코드가 전무한 레거시였기 때문에, **계층별 검증용 엔드포인트를 회귀 확인 기준선으로 먼저 구축**해 해당 계층만 검증한 뒤 다음으로 이동했습니다.

RESULT

Jakarta EE 전환부터 뷰 연동까지 전 구간을 원인 불명 오류로 인한 중단 없이 완료했습니다. 전환 중 발생한 호환성 이슈 53건은 Jakarta 패키지, Security 설정, MyBatis 바인딩, 뷰 경로, 빌드 의존성 유형으로 분류해 기록하며 해결했고, 서버 접속 없이 로그만으로 원인을 특정할 수 있게 되면서 버그 수정 주기가 **3일 → 1일**로 줄었습니다.

2. 데이터 접근 설계와 코드 구조 개선 — 일관성이나 효율이나

PROBLEM

신규 로그인 보안 기능을 설계하면서 세 가지 결정이 필요했습니다. 실패 이력이 여러 테이블에 분산된 구조를 어떻게 정리할지, 기존 MyBatis 외에 JPA를 도입할지, 그리고 레거시에서 이식한 코드의 필드 주입-계층 혼재 문제를 어떻게 정리할지였습니다.

ACTION

[테이블 및 인덱스 설계 최적화]

파편화되어 있던 기존 로그인 성공/실패 이력 테이블들을 **신규 단일 테이블로 구조화**하여 관리 효율을 높였습니다.

나아가 IP 차단 및 계정 잠금 로직에서 발생하는 **실제 환경의 WHERE 절 패턴을 전수 분석해 풀 테이블 스캔(Full Table Scan)을 방지할 복합 인덱스를 핵심 조회 경로별로 직접 설계**했습니다.

특히 인덱스 카디널리티와 조건절 유형을 함께 고려해 (IP, RESULT, LOG_DT)처럼 '**동등 조건(=) → 범위 조건(<, >)**' 순서로 컬럼을 배치했고, 대표 쿼리를 **EXPLAIN으로 확인해 풀 테이블 스캔이 Index Range Scan으로 바뀌는 것을 검증**했습니다.

[데이터 접근 방식 선택]

처음엔 일관성을 위해 MyBatis 단일 사용을 고려했지만, 로그인 이력처럼 단순 CRUD 위주 도메인에 MyBatis XML을 새로 작성하는 건 비효율적이었습니다.

결론은 "**복잡한 동적 쿼리는 MyBatis, 신규 단순 CRUD는 JPA**"로 분리하는 것이었고, 트레이드오프 이유를 문서로 남겼습니다.

다만 두 기술이 같은 DataSource와 트랜잭션 경계를 쓰도록 설정을 정리해야 했고, 예외 발생 시 양쪽 변경이 함께 롤백되는지 직접 테스트해 확인했습니다. 뷰 단계에서 예측하기 어려운 지연 로딩이 생기지 않도록 `open-in-view: false`로 조회 책임을 서비스 계층에 한정했습니다.

[코드 구조 개선]

필드 주입(@Autowired)을 생성자 주입으로 전환하자 기동 시점에 CompanyService ↔ UserExService 간 순환 참조가 즉시 드러났습니다. 지연 주입으로 덮는 대신, 두 서비스가 함께 쓰던 조회 책임을 분리해 호출 방향을 한쪽으로 정리하는 방식으로 해소했습니다. DI 방식이 스타일 차이가 아니라 **설계 문제를 드러내는 도구**라는 걸 직접 경험했습니다.

또한 단건 조회를 전제한 Repository 메서드가 운영 데이터에서 2건을 만나 **IncorrectResultSizeDataAccessException**이 발생했습니다. 우선 List 조회로 전환해 장애를 차단했고, 단건 전제가 깨진 원인인 중복 데이터 정리는 정렬 기준 명시와 유니크 제약 검토 과제로 분리해 기록했습니다. 이후 운영 이슈는 임시 조치와 근본 해결을 구분해 기록하고 있습니다.

RESULT

- 단일 테이블 통합 및 조회 패턴 기반 복합 인덱스 설계, 실행 계획(EXPLAIN) 검증으로 효과 확인
- MyBatis + JPA 하이브리드 전략으로 레거시 쿼리 자산 유지와 신규 개발 효율 양립
- 생성자 주입 전환으로 순환 참조 발굴·해소, 운영 예외는 임시 복구와 근본 과제로 구분해 대응

3. 운영 전환과 안정화 — 만든 것을 배포하고 운영까지 책임진다

- 운영 배포 완료(2026.06)**. 배포 일정과 영향 범위를 운영 담당자와 협의해 확정하고, 기존 톨킷 서비스는 Spring Boot 내장 Tomcat 기반으로 교체하되 앞단 Apache(AJP) 연동은 그대로 유지해 인프라 변경을 최소화했습니다. 현재 마이그레이션된 버전으로 서비스가 운영 중이며, 배포 중 발견된 레거시 잠재 결함(NPE, 리다이렉트 오류)도 즉시 수정해 반영했습니다.
- 사내 SSO(SAML) 인증 개편**. 인증 콜백 시 Session Fixation 방지용 세션 재생성으로 내부 검증 플러그가 함께 사라지는 문제를 찾아, 진입 경로 정분화와 콜백 시 플러그 재설정으로 해결했습니다.
- 운영 예외 분류 체계 정리**. 파일 다운로드 중 클라이언트 연결 종료로 발생하는 I/O 예외가 ERROR로 기록되어 모니터링 오경보를 만들던 것을, 전역 예외 핸들러에서 원인별로 분류해 해소했습니다.

핵심 성과

✔ 대규모 마이그레이션 완수 · 운영 배포

- Java 1.7 / 구형 Spring → Java 17 / Spring Boot 3.3.2
- Jakarta EE 전환부터 운영 배포(2026.06)까지 전 구간 1인 수행, 이후 운영까지 담당
- 폐쇄망 맞춤 로깅 전략 및 계층별 검증 체계 구축

✔ 신규 보안 로그인 시스템 설계 및 구현

- IP·사번 이중 차단 정책 설계, 조회 패턴 기반 복합 인덱스 설계
- MyBatis + JPA 하이브리드 구성, 트랜잭션 공존 직접 검증
- XML 인터셉터 → SecurityFilterChain 기반 커스텀 필터로 재설계

그 외 수행 업무 (2024.04 ~)

김가을 **후라이브러리 교체 프로젝트(2024~2025)** — API 통신 방식·서비스 로직 재설계, 앱 식별값 검증 로직 및 XSS-SQL Injection 방어 필터 서버단 구현

- 상시 운영** — 관리자·사용자 페이지 백엔드 기능 개발, 장애 대응 (Apache 액세스 로그·DB 쿼리 분석 기반 원인 추적)